السكربت المسموم في متصفح المظلوم

Lecture 6 — Cross-Site Scripting (XSS)

> **Cross-Site Scripting (XSS)** : the application puts untrusted input into a page **without neutralizing it**, so the browser runs it as **script** instead of showing it as text.

The browser can't tell a legitimate script from an injected one: both come from the same trusted origin with the same privileges. Same root cause as SQL injection (code vs data), but inside the **browser's** HTML/JS engine.

Category	Where the payload lives	Persistence	Typical delivery
Reflected	In the request itself (URL parameter)	Not stored; one response only	A crafted link, often by phishing
Stored	Saved on the server (comment, profile field)	**Persistent**; served to every visitor	No further interaction once posted
DOM-based	Never reaches the server; written by client-side script	Depends on the script	Manipulating a URL fragment or other client source

Q/ Which XSS category is the most consequential and why ?

- **Stored**: one successful injection runs in the browser of **every later visitor**, with no further action from the attacker.

DOM-based XSS: sources and sinks

> **Source** : where untrusted data **enters** client-side script.

> **Sink** : a function/property that turns a string into **parsed markup or executed code**.

A DOM XSS exists when data flows **from a source into a sink** without a safe transformation.

Sources	Sinks
location.hash, location.search	element.innerHTML, element.outerHTML

Sources	Sinks
document.referrer, document.URL	document.write(), document.writeln()
window.name	eval(), new Function()
postMessage data	setTimeout()/setInterval() with a **string**
	Event-handler attribute set from untrusted data

Safe alternative: **element.textContent** (always text) or build elements through the DOM. Server-side encoding can't reach DOM XSS because it never touches the server.

Q/ What does an attacker gain from XSS ?

1. **Session theft** (most common): read the session cookie and send it to the attacker's server → impersonate the victim without the password.
2. Capture **keystrokes**.
3. Redirect to a **fake login** form.
4. **Submit requests** on the victim's behalf (links to CSRF, Week 7).
5. **Deface** the page.

Defenses

> **Output encoding** : converting special characters (like < >) to harmless text **at the exact point** data is written into the page. The primary defense.

Output context	Where	Correct encoding
HTML body	Text between tags (inside p or div)	**HTML entity** encoding (< > & " ')
HTML attribute	Inside value="..." or href	Entity-encode **everything outside a small safe set** (attributes can be closed by more than >)
JavaScript variable	Inside a script, as an already-quoted string	**Unicode escaping**; only ever inside an existing quoted string
CSS value	Stylesheet or style attribute	**CSS hex** encoding

Output context	Where	Correct encoding
URL parameter	Query string / URL part	**Percent** encoding

Encoding must match its context. Wrong encoding = as exposed as none. Putting untrusted data **directly inside a script tag** (not in a quoted string) is a dangerous context that **no encoding fully fixes** — avoid it.

Q/ What are the additional layers of defense ?

1. **Input validation** with an **allow list** of permitted characters/formats (not a block list).
2. A **sanitization library** when some HTML must be accepted (parse and rebuild, no custom filters).
3. **Content-Security-Policy**: blocks inline scripts and limits script sources — limits damage even if injection slips through.
4. **HttpOnly** cookies: doesn't prevent XSS, but stops script reading the session cookie.

Frameworks

> **React** : values in `{curly braces}` are escaped automatically. Escape hatch: **dangerouslySetInnerHTML** (name is deliberate).

> **Angular** : **DomSanitizer** sanitizes HTML, style and URL bindings. Escape hatch: **bypassSecurityTrustHtml**.

Frameworks make the safe behaviour the **default**; using an escape hatch puts responsibility back on the developer.

Defense	Prevents XSS?	What it does
Output encoding	**Yes** (primary)	Data shown as text, not code
Input validation	Helps	Restricts what's accepted
CSP	Limits damage	Blocks inline / foreign scripts
HttpOnly	**No**	Protects the cookie from theft

> **OWASP placement** : XSS was its own category in older editions; **2021 merged it into A03:2021 Injection**, because it shares SQL injection's root cause (an interpreter that can't tell code from data) — only the interpreter differs (browser vs database).

Summary of Web Applications Security L6: Cross-Site Scripting (XSS). Built only from the lecture file; check the original slides for anything you're unsure about.